

Six Flags Project

David Watson, Matthew Hill, Caleb Short,
Cale Rosenbalm, & Luke Vaughan

Iteration Three

Iteration 3 - Temporary

Pattern Incorporation Reports

Decorator Pattern: Purchase Item

The decorator pattern is a structural design pattern that wraps an object in a decorator class to attach new or modified behaviors to it. This is especially useful when applying different combinations of added behaviors, so rather than having individualized classes for each combination, an object can simply be wrapped by any decorator that has relevant behavior.

The first iteration of purchase item simply listed all available items for a customer to buy, and had them select the desired item and the quantity. But taking one look at Six Flags ticket sales shows that there are far more purchase options. In addition to tickets, you can get fast passes, meal plans, etc. I found the decorator pattern to be the perfect solution to this problem. It also allowed me to further develop the Item class into several more useful abstractions and subtypes, such as Ticket and Season Pass.

The primary additions to the Purchase Item use case are all of the decorators and their AddOnService. For the purposes of this demo, I have implemented 4 wrappers, primarily focusing on Tickets and Season Passes, the primary business transaction of Six Flags. These are Fast Lanes, VIP Experience, Meal Plans, and Parking arrangements. All of these decorators, when applied to the object, will affect the price and benefits of the item. Meal Plans, in particular, also come in multiple types, which adjust the price based on the desired plan. PurchaseUI has also undergone major changes as a result of this refactor. When purchasing an item, the system will find any AddOns that apply to the given ItemType, and list them for the user to optionally add to their order. Any selections made will then be wrapped and subsequently added to the user's cart.

This new design is not only effective at applying benefits to what were once simple item objects, but it also gives Items much more room to expand. This could include new ItemTypes, ItemAddOns, discounts, bundles based on item combinations, and much more.

Builder Pattern: Refund Item

The Builder pattern separates the construction of a complex object from its representation, allowing the same construction process to produce different configurations of an object. This is a creational pattern because it decides how objects are assembled before they are used.

In the previous version of the refund system, processing a refund required passing three raw primitives directly through every layer of the application - processRefund (string recordId, int itemId, int quantity). This signature appeared in RefundControllerInterface, RefundSystemInterface, RefundController, RefundSystem, and RefundUI. There was no validation of these values until they reached inside RefundSystem, and there was no structured way to attach additional context like a reason or requester to the request. So adding a new field would have required changing every method signature across all five files.

Refactoring introduces RefundRequest in the models package, which serves as the Builder's product. It holds three required fields - recordId, itemId, and quantity with two optional fields reason and requestedBy. Its constructor is private meaning a RefundRequest can only be created through its inner Builder class. The builder exposes an interface where each field is set by a method call, and the final build() method validates that all required fields are present before the object is constructed. This guarantees that no invalid RefundRequest can ever reach the controller or system.

RefundUI is where the Builder is visibly used. As the user steps through the refund flow (example: selecting a record, selecting an item, entering a reason, entering a quantity) each piece of input maps directly to a Builder method call. Once all input is collected, build() is called and the resulting RefundRequest is handed to RefundController which passes it to RefundSystem for processing. RefundSystem simply unpacks the fields from the request object instead of receiving loose parameters.

This pattern improves the codebase in two ways. First, the interface signatures are simplified to a single RefundRequest parameter, making the contract between layers easier to extend off of and are cleaner to look at. Second, validation is centralized in one place with the Builder rather than scattered across multiple methods. If a new field like customerId or refundMethod is needed in the future, it can be added to the Builder as an optional field without changing any existing method signatures.

Strategy Pattern: Managing Customer Issues

The strategy pattern lets you define a family of algorithms, encapsulates each of them into their own class, and makes them interchangeable at runtime without the user needing to know which one is being used. This is a behavioral pattern because it governs how the program's objects delegate responsibility at runtime.

In the previous version of the HRController class, all resolution logic for customer issues was handled inside of a single switch statement inside of the updateIssue method. Each branch (Resolve, Document, Escalate) directly mutated the Issue object inline. It was functional, but it violated the Open/Closed Principle; adding a new action type would require modifying the HRController class directly, and the controller is responsible for knowing the details of every possible action.

Refactoring adds the IssueResolutionStrategy interface (inside of the interfaces package), which declares a single method resolve(Issue issue, String notes). Four strategy classes are added into a new package named Strategies that implement this interface: ResolveStrategy, DocumentStrategy, EscalateStrategy, and ReopenStrategy (reopening issues was added in while refactoring, the original code did not have this option). Each of these classes encapsulates one action's behavior. For example, EscalateStrategy sets the issue's status to "Escalated" and writes a resolution note onto said issue, while ResolveStrategy and DocumentStrategy apply the HR employee's entered notes directly.

The HRController then ties everything together. Instead of using the original switch statement, it keeps a map of action names to their corresponding strategy objects. When an HR employee submits an action to the system, the controller looks up the corresponding strategy and hands the work over to said strategy. The actions are no longer hardcoded into the controller class. If a new action type ever needed to be added in the future, the programmer would simply need to write a new strategy class and drop it into the map inside of HRController. Nothing else would need to change.

CustomerIssueUI is the client side of this strategy pattern. It just passes along the action that the user has input into the console. This class remains mostly unchanged from its original version.

Template Pattern: Hiring Employees

The template method pattern defines the basic steps of an algorithm in a parent class and allows subclasses to fill in specific steps without changing the overall order.

In this codebase, `HiringProcess` is the abstract class. Its `processHiringRequest` method is marked final, so the subclasses cannot change the order of the hiring steps. The method always validates the input, finds the applicant, checks that the employee ID is not already in use, finds the job opening, ensures the applicant belongs to that opening, runs the department-specific checks, creates the employee record, and then closes the job opening. This keeps the main hiring process in one place while still allowing each department subclass to add its own rules.

The main variation is `performDepartmentChecks(Applicant, JobOpening)`, which each department-specific subclass overrides. `RidesHiringProcess` checks that the job title is an operator or attendant role. `FoodServiceHiringProcess` checks for food service or kitchen roles, and `MaintenanceHiringProcess` checks for technician, engineer, or maintenance roles. `GeneralHiringProcess` is used for departments that do not need extra checks. There is also an optional hook, `getSuccessMessage()`, which lets each subclass add a department-specific message after a successful hire.

This pattern helps because the shared hiring logic is not copied across multiple classes. Common steps like validation, file updates, applicant status changes, and closing the job opening are handled once in `HiringProcess`. If another department is added later, we only need to create a new subclass with its own department check, rather than rewriting the entire hiring process. The UI uses `forDepartment(...)` to select the correct hiring process based on the selected job opening's department, so it doesn't need to know the details of each department's rules.

Observer Pattern: Manage Ride Wait Times

The observer pattern defines a one to many dependency between a subject and the observers, so when the subject changes the observers are notified and update automatically.

In the codebase, Ride is the subject. It maintains a list of RideObserver objects and calls notifyObservers() when setWaitTime() is called. The RideObserver interface defines a single update(String rideName, int newWaitTime) method that all observers must implement.

Two observers were created. WaitTimeDisplay prints a confirmation message to console whenever a rides wait time changes, giving the op an immediate visual feedback.

WaitTimeLogger writes a time stamped entry to wait_time_log.txt, creating a record of all changes.

The WaitTimeController registers both observers before calling setWaitTime(), which triggers the notification chain automatically. This means the controller does not need to manually handle display or logging, it instead just updates the tide and the observers respond on their own.

This pattern improves the separation of concerns in the codebase. Confirmation and logging logic can be removed from the controller entirely. If a new observer is needed in the future, it can be added without modifying Ride or WaitTimeController at all.

State Pattern Refactoring Report:

1. AI Software Used for Refactoring

Claude (Anthropic) was used as it provides direct codebase access through its project feature.

2. Prompt(s) Used

- a. We first got familiar with the pattern by asking:
"Explain what the State pattern is and how it applies to switching departments in a theme park management system."

Claude explained that the State pattern allows an object to alter its behavior when its internal state changes, and that each department could be represented as a concrete state class implementing a shared interface.

- b. We then gave Claude the full codebase and asked:
"Here is our Six Flags management system codebase across all three iterations and five team members. Integrate the State pattern for switching departments. The state should only be used for department switching."

Claude identified six departments from the existing codebase — Rides & Attractions, Food & Beverage, Security, Maintenance, Human Resources, and Guest Services — and proposed a DepartmentState interface with a concrete class for each.

- c. After receiving the structure we asked:
"Generate the DepartmentState interface, all concrete department state classes, and a DepartmentContext class. Then show how UserInterface would hold and switch between states."

Claude generated the interface, six concrete state classes, and a DepartmentContext class that holds the current state and exposes a switchDepartment() method.

3. Hand-Holding Required

Moderate hand-holding was required. Claude initially suggested applying the State pattern beyond department switching — specifically to ride status transitions (OPEN, CLOSED, MAINTENANCE) and refund status transitions (PURCHASED, PARTIAL_REFUND, REFUNDED). We redirected it per the assignment requirement that the State pattern be

Iteration 3 Refactoring Report

applied only to department switching. Claude complied and narrowed its output accordingly.

4. Outcome

The following was completed: a `DepartmentState` interface with `getDepartmentName()` and `activate()` methods in the `interfaces` package, a `DepartmentContext` class that holds the current state and exposes `switchDepartment()`, and six concrete state classes in a new `states` package — `RidesDepartmentState`, `FoodDepartmentState`, `SecurityDepartmentState`, `MaintenanceDepartmentState`, `HRDepartmentState`, and `GuestServicesDepartmentState` — each mapping to the relevant UI operations for that department. What was not completed was wiring `DepartmentContext` into `UserInterface`. The main menu still routes directly by number selection rather than delegating through the active department state.

5. Estimate of Remaining Work

We estimate 3 to 5 hours to complete the integration. The primary remaining work is updating `UserInterface` to hold a `DepartmentContext` and route menu selections through the active state. Each concrete state would need references to the relevant UI classes for its department — for example, `HRDepartmentState` would need references to `HREmployeeUI`, `SchedulingUI`, and `CustomerIssueUI`. This requires careful mapping across all five team members' use cases and risks breaking existing navigation if not done carefully. The core pattern structure is complete and correct. The remaining work is substitution and wiring.

6. Reflection

The State pattern for department switching is conceptually clean but arguably over-engineered for a project of this size. The existing switch statement in `UserInterface` is simple, readable, and works correctly. Replacing it with six state classes and a context object adds significant structural complexity for modest behavioral gain. That said, it is a defensible coding practice in the sense that adding a new department in the future would require only a new concrete state class rather than modifying the main menu switch directly, which aligns with the open-closed principle. Whether the outcome is worth the effort depends on the expected growth of the system — for a fixed six-department application, probably not. For a system expected to scale to many departments over time, the pattern pays off. We do consider it a form of defensive coding through refactoring, as it isolates department-specific behavior and reduces the risk of unintended side effects when the system changes.

Iteration 3 Refactoring Report – AI Output

DepartmentState.java

```
package interfaces;

public interface DepartmentState {
    String getDepartmentName();
    void activate();
}
```

DepartmentContext.java

```
package states;

import interfaces.DepartmentState;

public class DepartmentContext {
    private DepartmentState currentState;

    public DepartmentContext() {
        this.currentState = new RidesDepartmentState();
    }

    public void switchDepartment(DepartmentState newState) {
        System.out.println(" Switching from " +
            currentState.getDepartmentName()
                + " to " + newState.getDepartmentName());
        this.currentState = newState;
        this.currentState.activate();
    }

    public DepartmentState getCurrentState() {
        return currentState;
    }
}
```

Iteration 3 Refactoring Report – AI Output

```
    }

    public String getCurrentDepartmentName() {
        return currentState.getDepartmentName();
    }
}
```

RidesDepartmentState.java

```
package states;

import interfaces.DepartmentState;

public class RidesDepartmentState implements DepartmentState {

    @Override
    public String getDepartmentName() {
        return "Rides & Attractions";
    }

    @Override
    public void activate() {
        System.out.println("  Department activated: " +
            getDepartmentName());

        System.out.println("  Available operations: Ride
            Status, Ride Wait Times, Ride Maintenance");
    }
}
```

MaintenanceDepartmentState.java

```
package states;
```

Iteration 3 Refactoring Report – AI Output

```
import interfaces.DepartmentState;

public class MaintenanceDepartmentState implements
DepartmentState {

    @Override
    public String getDepartmentName() {
        return "Maintenance";
    }

    @Override
    public void activate() {
        System.out.println("  Department activated: " +
getDepartmentName());
        System.out.println("  Available operations: Ride
Maintenance");
    }
}
```

SecurityDepartmentState.java

```
package states;

import interfaces.DepartmentState;

public class SecurityDepartmentState implements
DepartmentState {

    @Override
    public String getDepartmentName() {
        return "Security";
    }
}
```

Iteration 3 Refactoring Report – AI Output

```
@Override
public void activate() {
    System.out.println("  Department activated: " +
getDepartmentName());
    System.out.println("  Available operations: Validate
Item");
}
}
```

FoodDepartmentState.java

```
package states;

import interfaces.DepartmentState;

public class FoodDepartmentState implements DepartmentState {

    @Override
    public String getDepartmentName() {
        return "Food & Beverage";
    }

    @Override
    public void activate() {
        System.out.println("  Department activated: " +
getDepartmentName());
        System.out.println("  Available operations: Purchase,
Inventory, Refund");
    }
}
```

Iteration 3 Refactoring Report – AI Output

GuestServicesDepartmentState.java

```
package states;

import interfaces.DepartmentState;

public class GuestServicesDepartmentState implements
DepartmentState {

    @Override
    public String getDepartmentName() {
        return "Guest Services";
    }

    @Override
    public void activate() {
        System.out.println("  Department activated: " +
getDepartmentName());
        System.out.println("  Available operations: Purchase,
Refund, Validate Item");
    }
}
```

HRDepartmentState.java

```
package states;

import interfaces.DepartmentState;

public class HRDepartmentState implements DepartmentState {

    @Override
    public String getDepartmentName() {
        return "Human Resources";
    }
}
```

Iteration 3 Refactoring Report – AI Output

```
@Override
public void activate() {
    System.out.println("  Department activated: " +
getDepartmentName());
    System.out.println("  Available operations: Hire
Employee, Employee Scheduling, Customer Issues");
}
}
```

Iteration Three

Software

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

ItemInterface.java

```
package interfaces;
import java.util.List;

public interface ItemInterface {
    String getName();
    double getPrice();
    List<String> getBenefits();

    String getInfo();
}
```

ItemType.java

```
package interfaces.types;
import interfaces.ItemInterface;
import models.item.Concession;
import models.item.DailyTicket;
import models.item.Merch;
import models.item.SeasonPass;

public enum ItemType {
    DAILY_TICKET {
        @Override public ItemInterface create(String name, double
price) { return new DailyTicket(name, price); }
    },

    SEASON_PASS {
        @Override public ItemInterface create(String name, double
price) { return new SeasonPass(name, price); }
    },

    CONCESSION {
        @Override public ItemInterface create(String name, double
price) { return new Concession(name, price); }
    },

    MERCH {
        @Override public ItemInterface create(String name, double
price) { return new Merch(name, price); }
    };

    public abstract ItemInterface create(String name, double price);
}
```

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

DailyTicket.java

```
package models.item;

import interfaces.types.ItemType;

public class DailyTicket extends Item {

    public DailyTicket(String name, double price) {
        super(name, price);
    }

    @Override
    public ItemType getType() {
        return ItemType.DAILY_TICKET;
    }
}
```

Concession.java

```
package models.item;

import interfaces.types.ItemType;

public class Concession extends Item {

    public Concession(String name, double price) {
        super(name, price);
    }

    @Override
    public ItemType getType() {
        return ItemType.CONCESSION;
    }
}
```

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

SeasonPass.java

```
package models.item;
import interfaces.types.ItemType;
import interfaces.types.SeasonPassType;

public class SeasonPass extends Item {
    private SeasonPassType type;
    private double discount;

    public SeasonPass(String name, double price) {
        super(name, price);
    }

    @Override
    public ItemType getType() {
        return ItemType.SEASON_PASS;
    }

    public SeasonPassType getSeasonPassType() {
        return type;
    }

    public void setSeasonPassType(SeasonPassType type) {
        this.type = type;
        discount = type.getDiscount();
        setBenefits(type.getBenefits());
    }
}
```

Merch.java

```
package models.item;
import interfaces.types.ItemType;

public class Merch extends Item {

    public Merch(String name, double price) {
        super(name, price);
    }

    @Override
    public ItemType getType() {
        return ItemType.MERCH;
    }
}
```

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

PurchaseController.java (partial)

```
public String addToCart(InventoryItem selectedItem, int quantity,
List<AddOnSelection> addOns) {
    if (!inventory.verifyQuantity(selectedItem, quantity)) {
        return "Insufficient quantity. Only " +
selectedItem.getQuantity() + " available.";
    }

    InventoryItem item = selectedItem;

    for (AddOnSelection addOn : addOns) {
        item = addOnService.apply(addOn.getAddOn(), item,
addOn.getConfig());
    }

    CartItem cartItem = factory.createCartItem(item.getItem(),
selectedItem, quantity);
    cartManager.addItem(cartItem);
    return null;
}
```

CartManager.java (partial)

```
public List<String> getCartInfo() {
    List<String> output = new ArrayList<>();

    if (cart.isEmpty()) {
        output.add("Cart is empty.");
        return output;
    }

    output.add("Cart:");

    for (int i = 0; i < cart.size(); i++) {
        CartItem cartItem = cart.get(i);

        output.add(" " + (i + 1) + ". " + cartItem.getInfo());
        for (String benefit : cartItem.getItem().getBenefits()) {
            output.add("    - " + benefit);
        }
    }

    return output;
}
```

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

ItemAddOns.java

```
package interfaces.types;

public enum ItemAddOns {
    FAST_LANE("Fast Lane", 125),
    VIP("VIP Experience", 350),
    PARKING("Parking", 39),
    MEAL_PLAN("Meal Plan", 0);

    private final String displayName;
    private final double price;

    ItemAddOns(String displayName, double price) {
        this.displayName = displayName;
        this.price = price;
    }

    public String getDisplayName() {
        return displayName;
    }

    public double getPrice() {
        return price;
    }
}
```

AddOnSelection.java

```
package models.item.decorators;
import interfaces.types.ItemAddOns;

public class AddOnSelection {
    private final ItemAddOns addOn;
    private final Object config; // MealPlanType or null

    public AddOnSelection(ItemAddOns addOn, Object config) {
        this.addOn = addOn;
        this.config = config;
    }

    public ItemAddOns getAddOn() { return addOn; }
    public Object getConfig() { return config; }
```

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

```
}
```

MealPlanType.java

```
package interfaces.types;
```

```
public enum MealPlanType {
    SINGLE_MEAL {
        @Override public double getPrice() { return 25; }
        @Override public String toString() { return "Single Meal"; }
    },
    ALL_DAY {
        @Override public double getPrice() { return 35; }
        @Override public String toString() { return "All Day Dining"; }
    },
    DAILY_DRINKS {
        @Override public double getPrice() { return 29; }
        @Override public String toString() { return "All Day Drinks"; }
    },
    ALL_SEASON {
        @Override public double getPrice() { return 145; }
        @Override public String toString() { return "All Season
Meals"; }
    },
    SEASON_DRINKS {
        @Override public double getPrice() { return 39; }
        @Override public String toString() { return "All Season
Drinks"; }
    };

    public abstract double getPrice();
}
```

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

AddOnService.java

```
package models.item.decorators;

import interfaces.ItemInterface;
import interfaces.types.ItemAddOns;
import interfaces.types.ItemType;
import interfaces.types.MealPlanType;
import models.item.InventoryItem;

public class AddOnService {

    public boolean supports(ItemAddOns addOn, ItemType type) {
        return switch (addOn) {
            case FAST_LANE, VIP, MEAL_PLAN ->
                type == ItemType.DAILY_TICKET || type ==
ItemType.SEASON_PASS;

            case PARKING ->
                type == ItemType.DAILY_TICKET;
        };
    }

    public InventoryItem apply(ItemAddOns addOn, InventoryItem item,
Object config) {
        ItemInterface wrapped = item.getItem();

        switch (addOn) {
            case FAST_LANE -> wrapped = new
FastLaneDecorator(wrapped);
            case VIP -> wrapped = new VIPDecorator(wrapped);
            case PARKING -> wrapped = new ParkingDecorator(wrapped);

            case MEAL_PLAN -> {
                MealPlanType type = (MealPlanType) config;
                wrapped = new MealPlanDecorator(wrapped, type);
            }
        }

        return new InventoryItem(
            wrapped,
            item.getType(),
            item.getLocation(),
            item.getQuantity()
        );
    }
}
```

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

```
}
```

ItemDecorator.java

```
package models.item.decorators;

import interfaces.ItemInterface;

import java.util.List;

public class ItemDecorator implements ItemInterface {
    protected ItemInterface wrappee;

    public ItemDecorator(ItemInterface item) {
        wrappee = item;
    }

    public String getName() { return wrappee.getName(); }
    public double getPrice() { return wrappee.getPrice(); }
    public List<String> getBenefits() { return wrappee.getBenefits(); }
}

    public String getInfo() { return wrappee.getInfo(); }
}
```

FastLaneDecorator.java

```
package models.item.decorators;

import interfaces.ItemInterface;
import java.util.ArrayList;
import java.util.List;

public class FastLaneDecorator extends ItemDecorator {

    public FastLaneDecorator(ItemInterface item) {
        super(item);
    }

    @Override
    public double getPrice() {
        return wrappee.getPrice() + 125;
    }
}
```

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

```
@Override
public List<String> getBenefits() {
    ArrayList<String> benefits = new
ArrayList<>(wrappee.getBenefits());
    benefits.add("Fast Lane ($125)");
    return benefits;
}
}
```

ParkingDecorator.java

```
package models.item.decorators;

import interfaces.ItemInterface;
import java.util.ArrayList;
import java.util.List;

public class ParkingDecorator extends ItemDecorator {

    public ParkingDecorator(ItemInterface item) {
        super(item);
    }

    @Override
    public double getPrice() {
        return wrappee.getPrice() + 39;
    }

    @Override
    public List<String> getBenefits() {
        ArrayList<String> benefits = new
ArrayList<>(wrappee.getBenefits());
        benefits.add("Parking ($39)");
        return benefits;
    }
}
```

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

VIPDecorator.java

```
package models.item.decorators;

import interfaces.ItemInterface;
import java.util.ArrayList;
import java.util.List;

public class VIPDecorator extends ItemDecorator {

    public VIPDecorator(ItemInterface item) {
        super(item);
    }

    @Override
    public double getPrice() {
        return wrappee.getPrice() + 350;
    }

    @Override
    public List<String> getBenefits() {
        ArrayList<String> benefits = new
ArrayList<>(wrappee.getBenefits());
        benefits.add("VIP Experience ($350)\n\t\t\t- Guided
Tour\n\t\t\t\t- Reserved Parking\n\t\t\t\t- Unlimited Drinks\n\t\t\t\t-
Front of Line Access");
        return benefits;
    }
}
```

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

MealPlanDecorator.java

```
package models.item.decorators;

import interfaces.ItemInterface;
import interfaces.types.MealPlanType;
import java.util.ArrayList;
import java.util.List;

public class MealPlanDecorator extends ItemDecorator {

    private final MealPlanType type;

    public MealPlanDecorator(ItemInterface item, MealPlanType type) {
        super(item);
        this.type = type;
    }

    @Override
    public double getPrice() {
        return wrappee.getPrice() + type.getPrice();
    }

    @Override
    public List<String> getBenefits() {
        ArrayList<String> benefits = new
ArrayList<>(wrappee.getBenefits());
        benefits.add("Meal Plan: " + type.toString() + " ($" +
type.getPrice() + ")");
        return benefits;
    }
}
```

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

PurchaseUI.java (partial)

```
private void showQuantityInput(InventoryItem item) {
    boolean entering = true;
    while (entering) {
        router.clearScreen();
        router.printBanner();
        System.out.println(ANSI_BOLD + "  Add to Cart" + ANSI_RESET);
        System.out.println("  " + "-".repeat(34));
        System.out.println("  Item:      " + item.getName());
        System.out.println("  Price:     $" + String.format("%.2f",
item.getPrice()));
        System.out.println("  Available: " + item.getQuantity());
        System.out.println("  " + "-".repeat(34));
        System.out.print("  Enter quantity (0 to cancel): ");

        String input = scanner.nextLine().trim();
        int quantity;
        try {
            quantity = Integer.parseInt(input);
            if (quantity == 0) return;
            if (quantity < 0) {
                System.out.println(ANSI_RED + "  Quantity must be
positive." + ANSI_RESET);
                router.pause();
                continue;
            }
        } catch (NumberFormatException e) {
            System.out.println(ANSI_RED + "  Invalid number." +
ANSI_RESET);
            router.pause();
            continue;
        }

        List<AddOnSelection> addOns = showAddOns(item);

        String error = controller.addToCart(item, quantity, addOns);
        if (error != null) {
            System.out.println(ANSI_RED + "  Error: " + error +
ANSI_RESET);
            router.pause();
        } else {
            System.out.println(ANSI_GREEN + "  Added to cart." +
ANSI_RESET);
            entering = false;
        }
    }
}
```

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

```
        showCartMenu();
    }
}

private List<AddOnSelection> showAddOns(InventoryItem item) {
    ArrayList<AddOnSelection> selected = new ArrayList<>();
    List<ItemAddOns> options = new ArrayList<>();

    for (ItemAddOns addOn : ItemAddOns.values()) {
        if (addOnService.supports(addOn, item.getType())) {
            options.add(addOn);
        }
    }

    if (options.isEmpty()) return selected;

    boolean selecting = true;
    while (selecting) {
        router.clearScreen();
        router.printBanner();

        System.out.println(ANSI_BOLD + "  Add-Ons for " +
            item.getName() + ANSI_RESET);
        System.out.println("  " + "-".repeat(40));
        for (int i = 0; i < options.size(); i++) {
            System.out.println("  " + (i + 1) + ". " +
                formatAddOn(options.get(i)));
        }

        System.out.println("  " + (options.size() + 1) + ". Done");
        System.out.print("  Select an option: ");

        String input = scanner.nextLine().trim();

        int choice;
        try {
            choice = Integer.parseInt(input);
        } catch (NumberFormatException e) {
            System.out.println(ANSI_RED + "  Invalid input." +
                ANSI_RESET);
            router.pause();
            continue;
        }
    }
}
```

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

```
        if (choice == options.size() + 1) {
            selecting = false;
        } else if (choice >= 1 && choice <= options.size()) {
            ItemAddOns selectedAddOn = options.get(choice - 1);

            if (!selected.contains(selectedAddOn)) {
                Object config = null;

                if (selectedAddOn == ItemAddOns.MEAL_PLAN) {
                    config = promptMealPlanType();
                }

                selected.add(new AddOnSelection(selectedAddOn,
config));

                System.out.println(ANSI_GREEN + "    Added " +
selectedAddOn.toString() + ANSI_RESET);
            } else {
                System.out.println(ANSI_RED + "    Already selected." +
ANSI_RESET);
            }

            router.pause();
        } else {
            System.out.println(ANSI_RED + "    Invalid selection." +
ANSI_RESET);
            router.pause();
        }
    }

    return selected;
}

private String formatAddOn(ItemAddOns addOn) {
    if (addOn == ItemAddOns.MEAL_PLAN) {
        return "Meal Plan (varies)";
    }

    return addOn.name().replace("_", " ")
        + " ($" + addOn.getPrice() + ")";
}

private MealPlanType promptMealPlanType() {
    MealPlanType[] types = MealPlanType.values();
```

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

```
while (true) {
    for (int i = 0; i < types.length; i++) {
        System.out.println((i + 1) + ". " + types[i] + " ($" +
types[i].getPrice() + ")");
    }

    System.out.print("\nSelect Meal Plan: ");

    try {
        int choice = Integer.parseInt(scanner.nextLine().trim());

        if (choice >= 1 && choice <= types.length) {
            return types[choice - 1];
        }
    } catch (Exception ignored) {}

    System.out.println("Invalid.");
}

}

public void showCartMenu() {
    boolean running = true;
    while (running) {
        router.clearScreen();
        router.printBanner();
        System.out.println(ANSI_BOLD + "  Cart" + ANSI_RESET);
        System.out.println("  " + "-".repeat(50));

        List<String> cartLines = controller.getCartInfo();
        for (String line : cartLines) {
            System.out.println("  " + line);
        }

        System.out.println("  " + "-".repeat(50));
        System.out.printf("   %-30s $%.2f%n", "Total:",
controller.calculateTotal());
        System.out.println("  " + "-".repeat(50));
        System.out.println("  1. Continue Shopping");
        System.out.println("  2. Remove Item");
        System.out.println("  3. Checkout");
        System.out.println("  4. Back to Main");
        System.out.println("  " + "-".repeat(50));
        System.out.print("  Select an option: ");

        String choice = scanner.nextLine().trim();
    }
}
```

Iteration 3 Refactoring - Purchase Item + Decorator Pattern

```
switch (choice) {
    case "1": showLocationSelection(); running = false; break;
    case "2": showRemoveItem(); break;
    case "3": showCheckout(); running = false; break;
    case "4": running = false; break;
    default:
        System.out.println(ANSI_RED + " Invalid option." +
ANSI_RESET);
        router.pause();
    }
}
```

Iteration 3 Refactoring - Refund Item + Builder Pattern

RefundRequest.java

```
package models;

/**
 * RefundRequest – the "Product" in the Builder pattern.
 *
 * Encapsulates all data needed to process a refund.
 * Constructed exclusively via RefundRequest.Builder to ensure
 * all required fields are present before the request is submitted.
 */
public class RefundRequest {

    // Required fields
    private final String recordId;
    private final int    itemIndex;
    private final int    quantity;
    private final String reason;
    private final String requestedBy;

    // Private constructor – only the Builder can call this
    private RefundRequest(Builder builder) {
        this.recordId    = builder.recordId;
        this.itemIndex   = builder.itemIndex;
        this.quantity    = builder.quantity;
        this.reason      = builder.reason;
        this.requestedBy = builder.requestedBy;
    }

    // — Getters

    public String getRecordId()    { return recordId; }
    public int    getItemIndex()   { return itemIndex; }
    public int    getQuantity()    { return quantity; }
    public String getReason()      { return reason; }
    public String getRequestedBy() { return requestedBy; }

    @Override
    public String toString() {
        return "RefundRequest{"
            + "recordId='" + recordId + '\''
            + ", itemIndex=" + itemIndex
            + ", quantity=" + quantity
            + ", reason='" + reason + '\''
            + ", requestedBy='" + requestedBy + '\''
            + '}';
    }
}
```

Iteration 3 Refactoring - Refund Item + Builder Pattern

```
}
```

```
// — Builder
```

```
/**
 * Builder for RefundRequest.
 *
 * Required: recordId, itemIndex, quantity
 * Optional: reason, requestedBy
 *
 * Usage:
 *   RefundRequest request = new RefundRequest.Builder()
 *       .recordId("REC-001")
 *       .itemIndex(0)
 *       .quantity(2)
 *       .reason("Wrong item purchased")
 *       .requestedBy("Guest")
 *       .build();
 */
public static class Builder {

    // Required
    private String recordId;
    private int    itemIndex = -1;
    private int    quantity  = -1;

    // Optional
    private String reason      = "";
    private String requestedBy = "Guest";

    public Builder recordId(String recordId) {
        this.recordId = recordId;
        return this;
    }

    public Builder itemIndex(int itemIndex) {
        this.itemIndex = itemIndex;
        return this;
    }

    public Builder quantity(int quantity) {
        this.quantity = quantity;
        return this;
    }
}
```

Iteration 3 Refactoring - Refund Item + Builder Pattern

```
public Builder reason(String reason) {
    this.reason = reason;
    return this;
}

public Builder requestedBy(String requestedBy) {
    this.requestedBy = requestedBy;
    return this;
}

/**
 * Validates required fields and constructs the RefundRequest.
 * Throws IllegalStateException if any required field is missing.
 */
public RefundRequest build() {
    if (recordId == null || recordId.isBlank()) {
        throw new IllegalStateException("RefundRequest requires a recordId.");
    }
    if (itemIndex < 0) {
        throw new IllegalStateException("RefundRequest requires a valid itemIndex.");
    }
    if (quantity <= 0) {
        throw new IllegalStateException("RefundRequest requires a quantity greater than zero.");
    }
    return new RefundRequest(this);
}
}
```

RefundControllerInterface.java

```
package interfaces;

import models.RefundRequest;

public interface RefundControllerInterface {
    String processRefund(RefundRequest request);
}
```

Iteration 3 Refactoring - Refund Item + Builder Pattern

RefundSystem.java

```
package controllers;

import data.ItemInventory;
import interfaces.PurchaseFileHandlerInterface;
import interfaces.RefundFileHandlerInterface;
import interfaces.RefundSystemInterface;
import models.Item;
import models.PurchaseRecord;
import models.RefundRequest;

import java.io.IOException;

public class RefundSystem implements RefundSystemInterface {
    private static final String SUCCESS_MESSAGE = "Refund processed successfully.";
    private static final String INVALID_DATA_MESSAGE = "Invalid refund data entered.";
    private static final String RECORD_NOT_FOUND = "No purchase record found.";
    private static final String ITEM_NOT_FOUND = "Item not found in purchase record.";
    private static final String NON_REFUNDABLE_MESSAGE = "Item cannot be refunded.";
    private static final String INVALID_QUANTITY = "Invalid quantity. Must be greater than zero and not exceed purchased amount.";
    private static final String FILE_ERROR_MESSAGE = "Unable to process refund because of a file error.";

    private final PurchaseFileHandlerInterface purchaseFileHandler;
    private final RefundFileHandlerInterface refundFileHandler;
    private final ItemInventory inventory;

    public RefundSystem(PurchaseFileHandlerInterface purchaseFileHandler,
                        RefundFileHandlerInterface refundFileHandler,
                        ItemInventory inventory) {
        this.purchaseFileHandler = purchaseFileHandler;
        this.refundFileHandler = refundFileHandler;
        this.inventory = inventory;
    }

    /**
     * Processes a refund using a RefundRequest built by
     * RefundRequest.Builder.
     * All required fields (recordId, itemIndex, quantity) are guaranteed

```

Iteration 3 Refactoring - Refund Item + Builder Pattern

```
        * non-null/valid by the Builder's build() method before reaching
here.
    */
    @Override
    public String processRefundRequest(RefundRequest request) {
        // Builder already validates required fields – just do a null
guard
        if (request == null) return INVALID_DATA_MESSAGE;

        String recordId = request.getRecordId();
        int    itemIndex = request.getItemIndex();
        int    quantity  = request.getQuantity();

        try {
            // Step 1: retrieve purchase record
            PurchaseRecord record =
purchaseFileHandler.findRecordById(recordId);
            if (record == null) return RECORD_NOT_FOUND;

            // Step 2: find item by index to avoid duplicate name
collision
            Item item = record.findItemByIndex(itemIndex);
            if (item == null) return ITEM_NOT_FOUND;

            // Step 3: check refundable
            if (!record.isRefundable(item.getName())) return
NON_REFUNDABLE_MESSAGE;

            // Step 4: validate quantity
            if (quantity <= 0 || quantity > item.getQuantity()) return
INVALID_QUANTITY;

            // Step 5: restore inventory
            inventory.increaseQuantity(item.getName(), item.getPrice(),
item.getLocation(), quantity);

            // Step 6: log refund
            if (!refundFileHandler.logRefund(recordId, item.getName(),
quantity)) {
                inventory.increaseQuantity(item.getName(),
item.getPrice(), item.getLocation(), -quantity);
                return FILE_ERROR_MESSAGE;
            }

            // Step 7: update item quantity in purchase record
            int newQuantity = item.getQuantity() - quantity;
```

Iteration 3 Refactoring - Refund Item + Builder Pattern

```
        purchaseFileHandler.updateItemQuantity(recordId, itemIndex,
newQuantity);

        // Step 8: update record status
        boolean allRefunded = record.getItems().stream()
            .allMatch(i -> i == item
                ? newQuantity == 0
                : i.getQuantity() == 0);
        String newStatus = allRefunded ? "REFUNDED" :
"PARTIAL_REFUND";
        purchaseFileHandler.updateRecordStatus(recordId, newStatus);

        return SUCCESS_MESSAGE;

    } catch (IOException e) {
        return FILE_ERROR_MESSAGE;
    }
}
```

RefundSystemInterface.java

```
package interfaces;

import models.RefundRequest;

public interface RefundSystemInterface {
    String processRefundRequest(RefundRequest request);
}
```

Iteration 3 Refactoring - Refund Item + Builder Pattern

RefundController.java

```
package controllers;

import interfaces.RefundControllerInterface;
import interfaces.RefundSystemInterface;
import models.RefundRequest;

public class RefundController implements RefundControllerInterface {
    private final RefundSystemInterface refundSystem;

    public RefundController(RefundSystemInterface refundSystem) {
        this.refundSystem = refundSystem;
    }

    /**
     * Accepts a fully-built RefundRequest (constructed via
     RefundRequest.Builder)
     * and delegates processing to the RefundSystem.
     */
    @Override
    public String processRefund(RefundRequest request) {
        return refundSystem.processRefundRequest(request);
    }
}
```

Iteration 3 Refactoring - Refund Item + Builder Pattern

RefundUI.java

```
package ui;

import controllers.RefundController;
import controllers.RefundSystem;
import data.ItemInventory;
import data.PurchaseFileHandler;
import data.RefundFileHandler;
import interfaces.RefundControllerInterface;
import models.Item;
import models.PurchaseRecord;
import models.RefundRequest;

import java.util.List;
import java.util.Scanner;

public class RefundUI {
    private static final String PURCHASE_FILE_PATH = "data/purchases.txt";
    private static final String REFUND_FILE_PATH = "data/refunds.txt";

    private static final String ANSI_BOLD = "\u001B[1m";
    private static final String ANSI_RESET = "\u001B[0m";
    private static final String ANSI_RED = "\u001B[31m";
    private static final String ANSI_GREEN = "\u001B[32m";

    private final RefundControllerInterface refundController;
    private final PurchaseFileHandler purchaseFileHandler;
    private final Scanner scanner;
    private final UserInterface router;

    public RefundUI(Scanner scanner, UserInterface router, ItemInventory
sharedInventory) {
        this.scanner = scanner;
        this.router = router;
        PurchaseFileHandler pfh = new
PurchaseFileHandler(PURCHASE_FILE_PATH);
        RefundFileHandler rfh = new
RefundFileHandler(REFUND_FILE_PATH);
        this.purchaseFileHandler = pfh;
        this.refundController = new RefundController(
            new RefundSystem(pfh, rfh, sharedInventory));
    }

    public void showRefundMenu() {
        boolean running = true;
        while (running) {
```

Iteration 3 Refactoring - Refund Item + Builder Pattern

```
router.clearScreen();
router.printBanner();
System.out.println(ANSI_BOLD + "  Process Item Refund" +
ANSI_RESET);
System.out.println("  " + "-".repeat(34));
System.out.println("  1. Process Refund");
System.out.println("  2. Back");
System.out.println("  " + "-".repeat(34));
System.out.print("  Select an option: ");

String choice = scanner.nextLine().trim();
switch (choice) {
    case "1": showRecordList(); break;
    case "2": running = false; break;
    default:
        System.out.println(ANSI_RED + "  Invalid option." +
ANSI_RESET);
        router.pause();
}
}

private void showRecordList() {
    boolean running = true;
    while (running) {
        router.clearScreen();
        router.printBanner();

        List<PurchaseRecord> records;
        try {
            records = purchaseFileHandler.readAllRecords();
        } catch (Exception e) {
            System.out.println(ANSI_RED + "  Could not load purchase
records." + ANSI_RESET);
            router.pause();
            return;
        }

        if (records.isEmpty()) {
            System.out.println("  No purchase records found.");
            router.pause();
            return;
        }

        System.out.println(ANSI_BOLD + "  Purchase Records" +
ANSI_RESET);
        System.out.println("  " + "-".repeat(65));
```

Iteration 3 Refactoring - Refund Item + Builder Pattern

```
        System.out.printf(" %-5s %-14s %-16s %-30s%n", "No.", "Record  
ID", "Status", "Items");  
        System.out.println(" " + "-".repeat(65));  
        for (int i = 0; i < records.size(); i++) {  
            PurchaseRecord record = records.get(i);  
            StringBuilder itemNames = new StringBuilder();  
            for (Item item : record.getItems()) {  
                if (itemNames.length() > 0) itemNames.append(", ");  
                itemNames.append(item.getName()).append("x").append(item.getQuantity());  
            }  
            System.out.printf(" %-5s %-14s %-16s %-30s%n",  
                (i + 1) + ".",  
                record.getRecordId(),  
                record.getStatus(),  
                itemNames.toString());  
        }  
        System.out.println(" " + (records.size() + 1) + ". Back");  
        System.out.println(" " + "-".repeat(65));  
        System.out.print(" Enter record number: ");  
  
        String choice = scanner.nextLine().trim();  
        int index;  
        try {  
            index = Integer.parseInt(choice) - 1;  
            if (index == records.size()) return;  
            if (index < 0 || index >= records.size()) {  
                System.out.println(ANSI_RED + " Invalid selection." +  
ANSI_RESET);  
                router.pause();  
                continue;  
            }  
        } catch (NumberFormatException e) {  
            System.out.println(ANSI_RED + " Invalid input." +  
ANSI_RESET);  
            router.pause();  
            continue;  
        }  
  
        showItemSelection(records.get(index));  
    }  
}  
  
private void showItemSelection(PurchaseRecord record) {  
    router.clearScreen();  
    router.printBanner();  
    List<Item> items = record.getItems();
```

Iteration 3 Refactoring - Refund Item + Builder Pattern

```
        System.out.println(ANSI_BOLD + "  Record: " + record.getRecordId()
+ ANSI_RESET);
        System.out.println("  " + "-".repeat(55));
        System.out.printf("  %-5s %-25s %-10s %-6s%n", "No.", "Name",
"Price", "Qty");
        System.out.println("  " + "-".repeat(55));
        for (int i = 0; i < items.size(); i++) {
            Item item = items.get(i);
            System.out.printf("  %-5s %-25s %-10s %-6s%n",
                (i + 1) + ".",
                item.getName(),
                String.format("%.2f", item.getPrice()),
                item.getQuantity());
        }
        System.out.println("  " + (items.size() + 1) + ". Cancel");
        System.out.println("  " + "-".repeat(55));
        System.out.print("  Select item to refund: ");

        String choice = scanner.nextLine().trim();
        int itemIndex;
        try {
            itemIndex = Integer.parseInt(choice) - 1;
            if (itemIndex == items.size()) return;
            if (itemIndex < 0 || itemIndex >= items.size()) {
                System.out.println(ANSI_RED + "  Invalid selection." +
ANSI_RESET);

                router.pause();
                return;
            }
        } catch (NumberFormatException e) {
            System.out.println(ANSI_RED + "  Invalid input." +
ANSI_RESET);

            router.pause();
            return;
        }

        showQuantityInput(record, itemIndex, items.get(itemIndex));
    }

    private void showQuantityInput(PurchaseRecord record, int itemIndex,
Item item) {
        boolean entering = true;
        while (entering) {
            router.clearScreen();
            router.printBanner();
            System.out.println(ANSI_BOLD + "  Refund Item" + ANSI_RESET);
```

Iteration 3 Refactoring - Refund Item + Builder Pattern

```
System.out.println(" " + "-".repeat(34));
System.out.println(" Item: " + item.getName());
System.out.println(" Location: " +
item.getLocation().name().replace("_", " "));
System.out.println(" Purchased Qty: " + item.getQuantity());
System.out.println(" " + "-".repeat(34));
System.out.print(" Enter reason for refund: ");
String reason = scanner.nextLine().trim();

System.out.print(" Enter quantity to refund (0 to cancel):
");

String input = scanner.nextLine().trim();
int quantity;
try {
    quantity = Integer.parseInt(input);
    if (quantity == 0) return;
} catch (NumberFormatException e) {
    System.out.println(ANSI_RED + " Invalid quantity." +
ANSI_RESET);

    router.pause();
    continue;
}

// — Builder pattern used here

// Instead of passing raw primitives, we construct a validated
// RefundRequest object step by step using
RefundRequest.Builder.
RefundRequest request;
try {
    request = new RefundRequest.Builder()
        .recordId(record.getRecordId())
        .itemIndex(itemIndex)
        .quantity(quantity)
        .reason(reason)
        .requestedBy("Guest")
        .build();
} catch (IllegalStateException e) {
    System.out.println(ANSI_RED + " " + e.getMessage() +
ANSI_RESET);

    router.pause();
    continue;
}

//

String result = refundController.processRefund(request);
```

Iteration 3 Refactoring - Refund Item + Builder Pattern

```
        if (result.contains("successfully")) {
            System.out.println(ANSI_GREEN + "    " + result +
ANSI_RESET);
        } else {
            System.out.println(ANSI_RED + "    " + result + ANSI_RESET);
        }
        router.pause();
        entering = false;
    }
}
```

Iteration 3 Refactoring - Customer Issues + Strategy Pattern

HRController.java

```
package controllers;

import data.IssueFileHandler;
import interfaces.IssueResolutionStrategy;
import models.Issue;
import strategies.DocumentStrategy;
import strategies.EscalateStrategy;
import strategies.ReopenStrategy;
import strategies.ResolveStrategy;

import java.io.IOException;
import java.util.List;
import java.util.Map;

public class HRController {
    private final IssueFileHandler issueFileHandler;
    private final Map<String, IssueResolutionStrategy> strategies =
Map.of(
    "Resolve", new ResolveStrategy(),
    "Document", new DocumentStrategy(),
    "Escalate", new EscalateStrategy(),
    "Reopen", new ReopenStrategy()
);

    public HRController(String issuesFilePath) {
        this.issueFileHandler = new IssueFileHandler(issuesFilePath);
    }

    public List<Issue> getIssues() throws IOException {
        return issueFileHandler.readIssues();
    }

    public Issue getIssueDetails(String issueID) throws IOException {
        return issueFileHandler.readIssue(issueID);
    }

    public boolean validateAction(String action) {
        return action != null && strategies.containsKey(action);
    }

    public void updateIssue(Issue issue, String action, String notes)
    {
        IssueResolutionStrategy strategy = strategies.get(action);
```

Iteration 3 Refactoring - Customer Issues + Strategy Pattern

```
        if (strategy != null) strategy.resolve(issue, notes);
    }

    public void createIssue(String type, String description) throws
IOException {
        List<Issue> issues = issueFileHandler.readIssues();
        int nextNum = issues.size() + 1;
        String newID = String.format("I%03d", nextNum);
        Issue issue = new Issue(newID, type, description, "Open",
        "");
        issueFileHandler.addIssue(issue);
    }

    public void saveIssue(Issue issue) throws IOException {
        issueFileHandler.writeIssue(issue);
    }
}
```

Issue.java (partial)

```
public void updateStatus(String status) { this.status = status; }
    public void addResolution(String notes) {
        if (resolutionNotes == null || resolutionNotes.isEmpty()) {
            resolutionNotes = notes;
        } else if (!notes.isEmpty()) {
            resolutionNotes = resolutionNotes + "; " + notes;
        }
    }
}
```

IssueResolutionStrategy.java

```
package interfaces;

import models.Issue;

public interface IssueResolutionStrategy {
    void resolve(Issue issue, String notes);
}
```

Iteration 3 Refactoring - Customer Issues + Strategy Pattern

DocumentStrategy.java

```
package strategies;

import interfaces.IssueResolutionStrategy;
import models.Issue;

public class DocumentStrategy implements IssueResolutionStrategy {
    @Override
    public void resolve(Issue issue, String notes) {
        issue.updateStatus("Documented");
        issue.addResolution(notes);
    }
}
```

EscalateStrategy.java

```
package strategies;

import interfaces.IssueResolutionStrategy;
import models.Issue;

public class EscalateStrategy implements IssueResolutionStrategy {
    @Override
    public void resolve(Issue issue, String notes) {
        issue.updateStatus("Escalated");
        issue.addResolution("Sent to appropriate department");
    }
}
```

Iteration 3 Refactoring - Customer Issues + Strategy Pattern

ReopenStrategy.java

```
package strategies;

import interfaces.IssueResolutionStrategy;
import models.Issue;

public class ReopenStrategy implements IssueResolutionStrategy {
    @Override
    public void resolve(Issue issue, String notes) {
        issue.updateStatus("Reopened");
        issue.addResolution(notes);
    }
}
```

ResolveStrategy.java

```
package strategies;

import interfaces.IssueResolutionStrategy;
import models.Issue;

public class ResolveStrategy implements IssueResolutionStrategy {
    @Override
    public void resolve(Issue issue, String notes) {
        issue.updateStatus("Resolved");
        issue.addResolution(notes);
    }
}
```

Iteration 3 Refactoring - Customer Issues + Strategy Pattern

CustomerIssueUI.java (partial)

```
public class CustomerIssueUI {
    ...

    // Step 1: HR selects manage customer issues
    public void showIssueMenu() {
        boolean running = true;
        while (running) {
            router.clearScreen();
            router.printBanner();
            System.out.println(ANSI_BOLD + "  Manage Customer Issues"
+ ANSI_RESET);
            System.out.println("  " + "-".repeat(34));
            System.out.println("  1. View Issues");
            System.out.println("  2. Add Issue");
            System.out.println("  3. Back");
            System.out.println("  " + "-".repeat(34));
            System.out.print("  Select an option: ");

            String choice = scanner.nextLine().trim();
            switch (choice) {
                case "1": showIssueList(); break;
                case "2": showAddIssue(); break;
                case "3": running = false; break;
                default:
                    System.out.println(ANSI_RED + "  Invalid option."
+ ANSI_RESET);

                    router.pause();
            }
        }
    }

    private void showAddIssue() {
        router.clearScreen();
        router.printBanner();
        System.out.println(ANSI_BOLD + "  Add Customer Issue" +
ANSI_RESET);
        System.out.println("  " + "-".repeat(34));
        System.out.println("  Types: Complaint, Incident, PR Case");
        System.out.println("  Enter C to cancel.");
        System.out.println("  " + "-".repeat(34));

        System.out.print("  Enter type: ");
        String type = scanner.nextLine().trim();
    }
}
```

Iteration 3 Refactoring - Customer Issues + Strategy Pattern

```
        if (isCancel(type)) return;
        if (!type.equalsIgnoreCase("Complaint") &&
!type.equalsIgnoreCase("Incident") && !type.equalsIgnoreCase("PR
Case")) {
            System.out.println(ANSI_RED + "  Invalid type. Must be
Complaint, Incident, or PR Case." + ANSI_RESET);
            router.pause();
            return;
        }

        System.out.print("  Enter description: ");
        String description = scanner.nextLine().trim();
        if (isCancel(description)) return;
        if (description.isEmpty()) {
            System.out.println(ANSI_RED + "  Description cannot be
empty." + ANSI_RESET);
            router.pause();
            return;
        }

        try {
            controller.createIssue(type, description);
            System.out.println(ANSI_GREEN + "  Issue added
successfully." + ANSI_RESET);
        } catch (IOException e) {
            System.out.println(ANSI_RED + "  Error saving issue: " +
e.getMessage() + ANSI_RESET);
        }
        router.pause();
    }

    // Step 2 & 3: retrieve and display issues
    private void showIssueList() {
        boolean running = true;
        while (running) {
            router.clearScreen();
            router.printBanner();

            List<Issue> issues;
            try {
                issues = controller.getIssues();
            } catch (IOException e) {
                System.out.println(ANSI_RED + "  Could not load
issues." + ANSI_RESET);
                router.pause();
            }
        }
    }
}
```

Iteration 3 Refactoring - Customer Issues + Strategy Pattern

```
        return;
    }

    // Alternate flow 2a: no issues found
    if (issues.isEmpty()) {
        System.out.println("  No issues available.");
        router.pause();
        return;
    }

    System.out.println(ANSI_BOLD + "  Customer Issues" +
ANSI_RESET);
    System.out.println("  " + "-".repeat(85));
    System.out.printf("   %-5s %-10s %-14s %-14s %-35s%n",
"No.", "Issue ID", "Type", "Status", "Description");
    System.out.println("  " + "-".repeat(85));
    for (int i = 0; i < issues.size(); i++) {
        Issue issue = issues.get(i);
        String desc = issue.getDescription().length() > 35
            ? issue.getDescription().substring(0, 32) + "..."
            : issue.getDescription();
        System.out.printf("   %-5s %-10s %-14s %-14s %-35s%n",
            (i + 1) + ".",
            issue.getIssueID(),
            issue.getType(),
            issue.getStatus(),
            desc);
    }
    System.out.println("  " + (issues.size() + 1) + ".
Back");

    System.out.println("  " + "-".repeat(85));
    System.out.print("  Select an issue: ");

    String choice = scanner.nextLine().trim();

    ...
```

Iteration 3 Refactoring - Customer Issues + Strategy Pattern

```
// Step 5 & 6: display issue details
private void showIssueDetails(Issue issue) {
    router.clearScreen();
    router.printBanner();
    System.out.println(ANSI_BOLD + "   Issue Details" +
ANSI_RESET);
    System.out.println("   " + "-".repeat(50));
    System.out.println("   ID:           " + issue.getIssueID());
    System.out.println("   Type:           " + issue.getType());
    System.out.println("   Status:         " + issue.getStatus());
    System.out.println("   Description: " +
issue.getDescription());
    System.out.println("   Resolution:   " +
(issue.getResolutionNotes().isEmpty()
    ? "None" : issue.getResolutionNotes()));
    System.out.println("   " + "-".repeat(50));

    boolean canReopen =
issue.getStatus().equalsIgnoreCase("Resolved")
    || issue.getStatus().equalsIgnoreCase("Documented")
    || issue.getStatus().equalsIgnoreCase("Escalated");
    if (canReopen) {
        System.out.println("   1. Reopen");
        System.out.println("   2. Back");
    } else {
        System.out.println("   1. Resolve");
        System.out.println("   2. Document");
        System.out.println("   3. Escalate");
        System.out.println("   4. Back");
    }
    System.out.println("   " + "-".repeat(50));
    System.out.print("   Select an action: ");

    String choice = scanner.nextLine().trim();

    if (isCancel(choice)) return;

    String action;
    if (canReopen) {
        switch (choice) {
            case "1": action = "Reopen"; break;
            case "2": return;
            default:

```

Iteration 3 Refactoring - Customer Issues + Strategy Pattern

```
                System.out.println(ANSI_RED + "  Invalid
selection." + ANSI_RESET);
                router.pause();
                return;
            }
        } else {
            switch (choice) {
                case "1": action = "Resolve"; break;
                case "2": action = "Document"; break;
                case "3": action = "Escalate"; break;
                case "4": return;
                default:
                    System.out.println(ANSI_RED + "  Invalid
selection." + ANSI_RESET);
                    router.pause();
                    return;
            }
        }
    }

    ...
}
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

HiringProcess.java

```
package controllers;

import interfaces.ApplicantFileHandlerInterface;
import interfaces.EmployeeFileHandlerInterface;
import interfaces.HiringSystemInterface;
import interfaces.JobOpeningFileHandlerInterface;
import models.Applicant;
import models.Employee;
import models.JobOpening;

import java.io.IOException;

public abstract class HiringProcess implements
HiringSystemInterface {

    private static final String INVALID_DATA_MESSAGE =
"Invalid hiring data entered.";
    private static final String APPLICANT_NOT_FOUND_MESSAGE =
"Applicant does not exist.";
    private static final String APPLICANT_UNAVAILABLE_MESSAGE =
"Applicant is not active.";
    private static final String APPLICANT_JOB_MISMATCH_MESSAGE =
    "Applicant is not associated with this job opening.";
    private static final String JOB_UNAVAILABLE_MESSAGE =
"Job opening is unavailable.";
    private static final String EMPLOYEE_EXISTS_MESSAGE =
"Employee ID already exists.";
    private static final String FILE_ERROR_MESSAGE =
"Unable to hire employee because of a file error.";

    protected final ApplicantFileHandlerInterface
applicantFileHandler;
    protected final EmployeeFileHandlerInterface
employeeFileHandler;
    protected final JobOpeningFileHandlerInterface
jobOpeningFileHandler;

    protected HiringProcess(ApplicantFileHandlerInterface
applicantFileHandler,
                           EmployeeFileHandlerInterface
employeeFileHandler,
                           JobOpeningFileHandlerInterface
jobOpeningFileHandler) {
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

```
this.applicantFileHandler = applicantFileHandler;
this.employeeFileHandler = employeeFileHandler;
this.jobOpeningFileHandler = jobOpeningFileHandler;
}

@Override
public final String processHiringRequest(String applicantId,
String employeeId, String jobOpeningId) {
    if (!validateHiringData(applicantId, employeeId,
jobOpeningId)) {
        return INVALID_DATA_MESSAGE;
    }

    applicantId = applicantId.trim();
    employeeId = employeeId.trim();
    jobOpeningId = jobOpeningId.trim();

    try {
        Applicant applicant =
applicantFileHandler.findApplicantById(applicantId);
        if (applicant == null) return
APPLICANT_NOT_FOUND_MESSAGE;
        if (!applicant.isActive()) return
APPLICANT_UNAVAILABLE_MESSAGE;

        if (employeeFileHandler.findEmployeeById(employeeId) !=
null) {
            return EMPLOYEE_EXISTS_MESSAGE;

            JobOpening jobOpening =
jobOpeningFileHandler.findJobOpeningById(jobOpeningId);
            if (jobOpening == null || !jobOpening.isAvailable()) {
                return JOB_UNAVAILABLE_MESSAGE;
            }
            if
(!applicant.getJobOpeningId().equals(jobOpening.getJobOpeningId())) {
                return APPLICANT_JOB_MISMATCH_MESSAGE;
            }

            String deptError = performDepartmentChecks(applicant,
jobOpening);
            if (deptError != null) return deptError;

            Employee employee = new Employee(employeeId,
applicant.getApplicantName(),
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

```
jobOpening.getDepartmentName());
        employeeFileHandler.writeEmployee(employee);
        if
(!jobOpeningFileHandler.updateJobOpeningStatus(jobOpeningId,
"CLOSED")

        ||
!applicantFileHandler.updateApplicantStatus(applicantId, "HIRED")) {
            return FILE_ERROR_MESSAGE;
        }
        return getSuccessMessage();
    } catch (IOException e) {
        return FILE_ERROR_MESSAGE;
    }
}

protected abstract String performDepartmentChecks(Applicant
applicant, JobOpening jobOpening);

protected String getSuccessMessage() {
    return "Employee hired successfully.";
}

public static HiringProcess forDepartment(String department,
        ApplicantFileHandlerInterface applicantFH,
        EmployeeFileHandlerInterface employeeFH,
        JobOpeningFileHandlerInterface jobFH) {
    if (department == null) return new
GeneralHiringProcess(applicantFH, employeeFH, jobFH);
    return switch (department.trim().toLowerCase()) {
        case "rides & attractions" -> new
RidesHiringProcess(applicantFH, employeeFH, jobFH);
        case "food & beverage"      -> new
FoodServiceHiringProcess(applicantFH, employeeFH, jobFH);
        case "maintenance"          -> new
MaintenanceHiringProcess(applicantFH, employeeFH, jobFH);
        default                      -> new
GeneralHiringProcess(applicantFH, employeeFH, jobFH);
    };
}

private boolean validateHiringData(String applicantId, String
employeeId, String jobOpeningId) {
    return !isBlank(applicantId) && !isBlank(employeeId) &&
!isBlank(jobOpeningId);
}
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

```
    }

    private boolean isBlank(String value) {
        return value == null || value.trim().isEmpty();
    }
}
```

GeneralHiringProcess.java

```
package controllers;

import interfaces.ApplicantFileHandlerInterface;
import interfaces.EmployeeFileHandlerInterface;
import interfaces.JobOpeningFileHandlerInterface;
import models.Applicant;
import models.JobOpening;

public class GeneralHiringProcess extends HiringProcess {

    public GeneralHiringProcess(ApplicantFileHandlerInterface
applicantFileHandler,
                                EmployeeFileHandlerInterface
employeeFileHandler,
                                JobOpeningFileHandlerInterface
jobOpeningFileHandler) {
        super(applicantFileHandler, employeeFileHandler,
jobOpeningFileHandler);
    }

    @Override
    protected String performDepartmentChecks(Applicant applicant,
JobOpening jobOpening) {
        return null;
    }
}
```

FoodServiceHiringProcess.java

```
package controllers;

import interfaces.ApplicantFileHandlerInterface;
import interfaces.EmployeeFileHandlerInterface;
import interfaces.JobOpeningFileHandlerInterface;
import models.Applicant;
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

```
import models.JobOpening;

public class FoodServiceHiringProcess extends HiringProcess {

    public FoodServiceHiringProcess(ApplicantFileHandlerInterface
applicantFileHandler,
                                     EmployeeFileHandlerInterface
employeeFileHandler,
                                     JobOpeningFileHandlerInterface
jobOpeningFileHandler) {
        super(applicantFileHandler, employeeFileHandler,
jobOpeningFileHandler);
    }

    @Override
    protected String performDepartmentChecks(Applicant applicant,
JobOpening jobOpening) {
        String title = jobOpening.getJobTitle().toLowerCase();
        if (!title.contains("food") && !title.contains("service")
&& !title.contains("kitchen")) {
            return "Food & Beverage positions must be food service
or kitchen roles.";
        }
        return null;
    }

    @Override
    protected String getSuccessMessage() {
        return "Employee hired successfully. Food handler
certification must be obtained within 14 days.";
    }
}
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

MaintenanceHiringProcess.java

```
package controllers;

import interfaces.ApplicantFileHandlerInterface;
import interfaces.EmployeeFileHandlerInterface;
import interfaces.JobOpeningFileHandlerInterface;
import models.Applicant;
import models.JobOpening;

public class MaintenanceHiringProcess extends HiringProcess {

    public MaintenanceHiringProcess(ApplicantFileHandlerInterface
applicantFileHandler,

                                   EmployeeFileHandlerInterface
employeeFileHandler,

                                   JobOpeningFileHandlerInterface
jobOpeningFileHandler) {
        super(applicantFileHandler, employeeFileHandler,
jobOpeningFileHandler);
    }

    @Override
    protected String performDepartmentChecks(Applicant applicant,
JobOpening jobOpening) {
        String title = jobOpening.getJobTitle().toLowerCase();
        if (!title.contains("technician") &&
!title.contains("engineer") && !title.contains("maintenance")) {
            return "Maintenance positions must be technician or
engineer roles.";
        }
        return null;
    }

    @Override
    protected String getSuccessMessage() {
        return "Employee hired successfully. Technical safety
induction must be scheduled within 7 days.";
    }
}
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

RidesHiringProcess.java

```
package controllers;

import interfaces.ApplicantFileHandlerInterface;
import interfaces.EmployeeFileHandlerInterface;
import interfaces.JobOpeningFileHandlerInterface;
import models.Applicant;
import models.JobOpening;

public class RidesHiringProcess extends HiringProcess {

    public RidesHiringProcess(ApplicantFileHandlerInterface
applicantFileHandler,
                                EmployeeFileHandlerInterface
employeeFileHandler,
                                JobOpeningFileHandlerInterface
jobOpeningFileHandler) {
        super(applicantFileHandler, employeeFileHandler,
jobOpeningFileHandler);
    }

    @Override
    protected String performDepartmentChecks(Applicant applicant,
JobOpening jobOpening) {
        String title = jobOpening.getJobTitle().toLowerCase();
        if (!title.contains("operator") &&
!title.contains("attendant")) {
            return "Rides & Attractions positions must be operator
or attendant roles.";
        }
        return null;
    }

    @Override
    protected String getSuccessMessage() {
        return "Employee hired successfully. Safety certification
must be completed within 30 days.";
    }
}
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

ApplicantFileHandler.java

```
package data;

import interfaces.ApplicantFileHandlerInterface;
import models.Applicant;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.util.ArrayList;
import java.util.List;

public class ApplicantFileHandler implements
ApplicantFileHandlerInterface {
    private final Path applicantFilePath;

    public ApplicantFileHandler(String applicantFilePath) {
        this.applicantFilePath = Paths.get(applicantFilePath);
    }

    @Override
    public Applicant findApplicantById(String applicantId) throws
IOException {
        for (Applicant applicant : readAllApplicants()) {
            if (applicant.getApplicantId().equals(applicantId)) {
                return applicant;
            }
        }
        return null;
    }

    @Override
    public boolean updateApplicantStatus(String applicantId, String
applicantStatus) throws IOException {
        List<Applicant> applicants = readAllApplicants();
        boolean updated = false;

        for (int i = 0; i < applicants.size(); i++) {
            Applicant applicant = applicants.get(i);
            if (applicant.getApplicantId().equals(applicantId)) {
                applicants.set(i,
applicant.withStatus(applicantStatus));
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

```
        updated = true;
        break;
    }
}

if (updated) {
    writeAllApplicants(applicants);
}
return updated;
}

@Override
public List<Applicant> readAllApplicants() throws IOException {
    List<Applicant> applicants = new ArrayList<>();
    if (Files.notExists(applicantFilePath)) return applicants;
    for (String line : Files.readAllLines(applicantFilePath)) {
        if (line.trim().isEmpty()) continue;
        Applicant applicant = parseApplicantRecord(line);
        if (applicant != null) applicants.add(applicant);
    }
    return applicants;
}

private Applicant parseApplicantRecord(String line) {
    String[] parts = line.split("\\|");
    if (parts.length == 2) {
        return new Applicant(parts[0].trim(), parts[1].trim());
    }
    if (parts.length == 3) {
        return new Applicant(parts[0].trim(), parts[1].trim(),
parts[2].trim());
    }
    if (parts.length >= 4) {
        return new Applicant(parts[0].trim(), parts[1].trim(),
parts[2].trim(), parts[3].trim());
    }
    return null;
}

private void writeAllApplicants(List<Applicant> applicants)
throws IOException {
    if (applicantFilePath.getParent() != null) {
        Files.createDirectories(applicantFilePath.getParent());
    }
    StringBuilder contents = new StringBuilder();
}
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

```
        for (Applicant applicant : applicants) {

contents.append(formatApplicantRecord(applicant)).append(System.lineS
eparator());
        }
        Files.writeString(applicantFilePath, contents.toString(),
            StandardOpenOption.CREATE,
StandardOpenOption.TRUNCATE_EXISTING);
    }

    @Override
    public List<Applicant> findApplicantsByJobOpeningId(String
jobOpeningId) throws IOException {
        List<Applicant> result = new ArrayList<>();
        for (Applicant applicant : readAllApplicants()) {
            if (applicant.getJobOpeningId().equals(jobOpeningId)) {
                result.add(applicant);
            }
        }
        return result;
    }

    private String formatApplicantRecord(Applicant applicant) {
        return applicant.getApplicantId() + " | " +
applicant.getApplicantName() + " | "
            + applicant.getApplicantStatus().toUpperCase() + " | "
+ applicant.getJobOpeningId();
    }
}
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

ApplicantFileHandlerInterface.java

```
package interfaces;

import models.Applicant;

import java.io.IOException;
import java.util.List;

public interface ApplicantFileHandlerInterface {
    Applicant findApplicantById(String applicantId) throws
IOException;
    boolean updateApplicantStatus(String applicantId, String
applicantStatus) throws IOException;
    List<Applicant> readAllApplicants() throws IOException;
    List<Applicant> findApplicantsByJobOpeningId(String
jobOpeningId) throws IOException;
}
```

Applicant.java

```
package models;

public class Applicant {
    private final String applicantId;
    private final String applicantName;
    private final String applicantStatus;
    private final String jobOpeningId;

    public Applicant(String applicantId, String applicantName) {
        this(applicantId, applicantName, "ACTIVE", "");
    }

    public Applicant(String applicantId, String applicantName,
String applicantStatus) {
        this(applicantId, applicantName, applicantStatus, "");
    }

    public Applicant(String applicantId, String applicantName,
String applicantStatus, String jobOpeningId) {
        this.applicantId      = applicantId;
        this.applicantName    = applicantName;
        this.applicantStatus  = applicantStatus;
        this.jobOpeningId     = jobOpeningId == null ? "" :
jobOpeningId;
    }
}
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

```
public String get ApplicantId()      { return applicantId; }
public String get ApplicantName()    { return applicantName; }
public String get ApplicantStatus() { return applicantStatus; }
public String get JobOpeningId()     { return jobOpeningId; }

public boolean isActive() {
    return "ACTIVE".equalsIgnoreCase(applicantStatus);
}

public Applicant withStatus(String applicantStatus) {
    return new Applicant(applicantId, applicantName,
applicantStatus, jobOpeningId);
}

@Override
public String toString() {
    return "Applicant{id='" + applicantId + "', name='" +
applicantName
        + "', status='" + applicantStatus + "',
jobOpeningId='" + jobOpeningId + "'}";
}
}
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

HREmployeeUI.java

```
package ui;

import controllers.HiringController;
import controllers.HiringProcess;
import data.ApplicantFileHandler;
import data.EmployeeFileHandler;
import data.JobOpeningFileHandler;
import interfaces.ApplicantFileHandlerInterface;
import interfaces.EmployeeFileHandlerInterface;
import interfaces.HiringControllerInterface;
import interfaces.JobOpeningFileHandlerInterface;
import models.Applicant;
import models.JobOpening;

import java.util.List;
import java.util.Scanner;

public class HREmployeeUI {
    private static final String APPLICANT_FILE_PATH =
"data/applicants.txt";
    private static final String EMPLOYEE_FILE_PATH =
"data/employees.txt";
    private static final String JOB_OPENING_FILE_PATH =
"data/job_openings.txt";

    private static final String ANSI_BOLD = "\u001B[1m";
    private static final String ANSI_RESET = "\u001B[0m";
    private static final String ANSI_RED = "\u001B[31m";
    private static final String ANSI_GREEN = "\u001B[32m";

    private final ApplicantFileHandlerInterface
applicantFileHandler;
    private final EmployeeFileHandlerInterface
employeeFileHandler;
    private final JobOpeningFileHandlerInterface
jobOpeningFileHandler;
    private final UserInterface router;
    private final Scanner scanner;

    public HREmployeeUI(UserInterface router, Scanner scanner) {
        this.router = router;
        this.scanner = scanner;
    }
}
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

```
        this.applicantFileHandler = new
ApplicantFileHandler(APPLICANT_FILE_PATH);
        this.employeeFileHandler = new
EmployeeFileHandler(EMPLOYEE_FILE_PATH);
        this.jobOpeningFileHandler = new
JobOpeningFileHandler(JOB_OPENING_FILE_PATH);
    }

    public void startHiringProcess() {
        boolean running = true;
        while (running) {
            router.clearScreen();
            router.printBanner();
            System.out.println(ANSI_BOLD + "  HR -- Hire Employee"
+ ANSI_RESET);
            System.out.println("  " + "-".repeat(34));
            System.out.println("  1. Hire Employee");
            System.out.println("  2. Back");
            System.out.println("  " + "-".repeat(34));
            System.out.print("  Select an option: ");

            String choice = scanner.nextLine().trim();
            switch (choice) {
                case "1": selectJobOpening(); break;
                case "2": running = false;      break;
                default:
                    System.out.println(ANSI_RED + "  Invalid
option." + ANSI_RESET);
                    router.pause();
            }
        }
    }

    private void selectJobOpening() {
        while (true) {
            router.clearScreen();
            router.printBanner();
            System.out.println(ANSI_BOLD + "  Hire Employee -
Select Job Opening" + ANSI_RESET);
            System.out.println("  " + "-".repeat(34));

            showAvailableJobOpenings();

            System.out.println();
            System.out.println("  Enter C to cancel.");
        }
    }
}
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

```
        System.out.println("  " + "-".repeat(34));

        String jobOpeningId = readLine("  Job Opening ID: ");
        if (isCancel(jobOpeningId)) return;

        JobOpening jobOpening;
        try {
            jobOpening =
jobOpeningFileHandler.findJobOpeningById(jobOpeningId);
        } catch (Exception e) {
            displayResult("Unable to load job opening.");
            return;
        }

        if (jobOpening == null || !jobOpening.isAvailable()) {
            System.out.println(ANSI_RED + "  Job opening is
unavailable." + ANSI_RESET);
            router.pause();
            continue;
        }

        promptForApplicant(jobOpening);
        return;
    }

    private void promptForApplicant(JobOpening jobOpening) {
        HiringProcess process = HiringProcess.forDepartment(
            jobOpening.getDepartmentName(), applicantFileHandler,
employeeFileHandler, jobOpeningFileHandler);
        HiringControllerInterface controller = new
HiringController(process);

        while (true) {
            router.clearScreen();
            router.printBanner();
            System.out.println(ANSI_BOLD + "  Hire Employee - " +
jobOpening.getJobTitle()
                + " (" + jobOpening.getDepartmentName() + ")" +
ANSI_RESET);
            System.out.println("  " + "-".repeat(34));

            showApplicantsForJobOpening(jobOpening.getJobOpeningId());
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

```
        System.out.println();
        System.out.println("    Enter C at any prompt to
cancel.");
        System.out.println("    " + "-".repeat(34));

        String applicantId = readLine("    Applicant ID: ");
        if (isCancel(applicantId)) return;

        String employeeId = readLine("    New Employee ID: ");
        if (isCancel(employeeId)) return;

        String result = controller.hireEmployee(applicantId,
employeeId, jobOpening.getJobOpeningId());
        displayResult(result);

        if (result.contains("successfully")
            || result.equals("Applicant does not exist.")
            || result.equals("Unable to hire employee
because of a file error.)) {
            return;
        }
    }
}

    public void displayResult(String message) {
        System.out.println();
        if (message.contains("successfully")) {
            System.out.println(ANSI_GREEN + "    " + message +
ANSI_RESET);
        } else {
            System.out.println(ANSI_RED + "    " + message +
ANSI_RESET);
        }
        router.pause();
    }

    private void showApplicantsForJobOpening(String jobOpeningId) {
        List<Applicant> applicants;
        try {
            applicants =
applicantFileHandler.findApplicantsByJobOpeningId(jobOpeningId);
        } catch (Exception e) {
            System.out.println(ANSI_RED + "    Could not load
applicants." + ANSI_RESET);
            return;
        }
    }
}
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

```
}

System.out.println();
System.out.println(" " + "-".repeat(50));
System.out.printf(" %-12s %-24s %-10s%n", "ID", "Name",
"Status");
System.out.println(" " + "-".repeat(50));
boolean found = false;
for (Applicant applicant : applicants) {
    if (applicant.isActive()) {
        System.out.printf(" %-12s %-24s %-10s%n",
            applicant.getApplicantId(),
            applicant.getApplicantName(),
            applicant.getApplicantStatus());
        found = true;
    }
}
if (!found) System.out.println(" No active applicants
found for this opening.");
}

private void showAvailableJobOpenings() {
    List<JobOpening> jobOpenings;
    try {
        jobOpenings =
jobOpeningFileHandler.readAllJobOpenings();
    } catch (Exception e) {
        System.out.println(ANSI_RED + " Could not load job
openings." + ANSI_RESET);
        return;
    }

    System.out.println();
    System.out.println(ANSI_BOLD + " Available Job Openings" +
ANSI_RESET);
    System.out.println(" " + "-".repeat(70));
    System.out.printf(" %-12s %-28s %-22s %-10s%n", "ID",
"Title", "Department", "Status");
    System.out.println(" " + "-".repeat(70));
    boolean found = false;
    for (JobOpening jobOpening : jobOpenings) {
        if (jobOpening.isAvailable()) {
            System.out.printf(" %-12s %-28s %-22s %-10s%n",
                jobOpening.getJobOpeningId(),
                jobOpening.getJobTitle(),
```

Iteration 3 Refactoring - Hiring Employees + Template Pattern

```
                jobOpening.getDepartmentName(),
                jobOpening.getOpeningStatus());
        found = true;
    }
}
if (!found) System.out.println("  No available job openings
found.");
}

private String readLine(String prompt) {
    System.out.print(prompt);
    return scanner.nextLine().trim();
}

private boolean isCancel(String value) {
    return value.equalsIgnoreCase("C") ||
value.equalsIgnoreCase("CANCEL");
}
}
```

Iteration 3 Refactoring - Ride Wait Times + Observer Pattern

WaitTimeController.java (partial)

```
public String updateWaitTime(String rideID, int waitTime) {
    if (waitTime < 0) return "Wait time cannot be negative.";

    List<Ride> rides = loadRides();
    for (Ride r : rides) {
        if (r.getRideID().equals(rideID)) {
            if (!r.getStatus().equals("OPEN"))
                return "Cannot update wait time – ride is not
currently OPEN.";

            // Register observers before updating
            r.addObserver(new WaitTimeDisplay());
            r.addObserver(new WaitTimeLogger());

            r.setWaitTime(waitTime); // this triggers
notifyObservers()
            saveRides(rides);
            return null; // success
        }
    }
    return "Ride not found in records.";
}
```

RideObserver.java

```
package interfaces;

public interface RideObserver {
    void update(String rideName, int newWaitTime);
}
```

Iteration 3 Refactoring - Ride Wait Times + Observer Pattern

Ride.java (partial)

```
public class Ride {
    ...

    private List<RideObserver> observers = new ArrayList<>();

    ...

    public void addObserver(RideObserver observer) {
        observers.add(observer);
    }

    public void removeObserver(RideObserver observer) {
        observers.remove(observer);
    }

    private void notifyObservers() {
        for (RideObserver o : observers) {
            o.update(name, waitTime);
        }
    }

    ...

    public void setWaitTime(int waitTime) {
        this.waitTime = waitTime;
        notifyObservers();
    }
}
```

Iteration 3 Refactoring - Ride Wait Times + Observer Pattern

WaitTimeDisplay.java

```
package models;

import interfaces.RideObserver;

public class WaitTimeDisplay implements RideObserver {
    @Override
    public void update(String rideName, int newWaitTime) {
        System.out.println("[DISPLAY] " + rideName +
            " wait time updated to " + newWaitTime + " minutes.");
    }
}
```

WaitTimeLogger.java

```
package models;

import interfaces.RideObserver;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDateTime;

public class WaitTimeLogger implements RideObserver {
    private static final String LOG_FILE = "six-flags/Six
Flags/data/wait_time_log.txt";

    @Override
    public void update(String rideName, int newWaitTime) {
        try (FileWriter fw = new FileWriter(LOG_FILE, true)) {
            fw.write(rideName + " wait time set to " + newWaitTime +
                " minutes at " + LocalDateTime.now() + "\n");
        } catch (IOException e) {
            System.out.println("Error writing wait time log: " +
e.getMessage());
        }
    }
}
```